https://help.accuknox.com/secrets-manager/overview/#check-the-security-api-and-support-terms

Check the Security, API and Support Terms

Security and Encryption

How the seal, Shamir's Secret Sharing, and encryption at rest and in transit protect each secret.

API REST API Reference

Base URL, authentication, the main endpoints, and the OpenAPI document from your own server.

Multi-Region HA and DR

One cluster per region, snapshot replication, and the failover runbook.

Support and SLA

Severity levels, response targets, and what to send in a ticket.

https://help.accuknox.com/secrets-manager/architecture/#three-nodes-share-one-encrypted-store

Three Nodes Share One Encrypted Store

In high-availability mode, three Secrets Manager pods run as a StatefulSet. One pod is active and the other two stand by. Each pod keeps its own copy of the data on a Raft integrated storage disk, 10Gi by default, and the copies replicate between the pods.

There is no separate database to run or back up. Secrets Manager encrypts data before it reaches disk. See Security and Encryption for the key chain behind that encryption, and High Availability and Backup for failover within a region and across regions.

https://help.accuknox.com/secrets-manager/deployment/#choose-a-deployment-model

Choose a Deployment Model

Run Secrets Manager yourself with the Helm chart on this page, or let AccuKnox run a managed instance. Every managed instance runs in high availability mode and is hardened with AccuKnox CWPP.

	Managed shared instance	Managed dedicated instance	Self-hosted
Fits	Non-production workloads	Production	Production, on-premises or air-gapped
Namespaces	One, more at extra cost	Several	Your choice
Backups	Daily	At the frequency you set	Snapshots every 15 minutes, once you turn them on
API request limits	Yes	No	Set by you
Audit logs	Every interaction logged	On, with a direct feed to your AccuKnox SIEM and CDR subscription	Turn on `server.auditStorage`
Location	North America, Europe, the Middle East or India	On-premises or air-gapped is available	Your cluster
Run by	AccuKnox	AccuKnox	Your team

Source: AccuKnox Secrets Management. The rest of this guide covers the self-hosted install.

https://help.accuknox.com/secrets-manager/deployment/#prerequisites

Prerequisites

System requirements | Kubernetes requirements | **Production requirements**

Requirement	Details
Worker nodes	3, one Secrets Manager pod per host, ideally one per availability zone
Storage per pod	10Gi for Raft data, plus 10Gi for the audit log when `server.auditStorage` is on
Network ports	8200 for the API and UI. 8201 between Secrets Manager pods, for Raft and request forwarding
TLS certificate	A certificate and key for the 8200 listener, trusted by every client. The listener accepts TLS 1.2 or later by default. See TLS Protects Every Connection
Snapshot bucket	An S3-compatible bucket, plus a Kubernetes secret with its access key
Key holders	5 people to hold the unseal key shares. Any 3 unseal the server
Second region	Optional, for disaster recovery. See High Availability and Backup

Read Security and Encryption before you choose between key holders and auto-unseal.

https://help.accuknox.com/secrets-manager/security/

Security and Encryption

AccuKnox Secrets Manager encrypts every secret with AES-256 in GCM mode before it writes the secret to disk. The server cannot decrypt anything until 3 of 5 key holders unseal it, and it never stores the key that unlocks the data.

Encrypted at rest

AES-256-GCM with a random 96-bit nonce for every object. A changed byte fails the integrity check.

Sealed by default

Shamir's Secret Sharing splits the unseal key into 5 shares. Any 3 rebuild it.

Encrypted in transit

TLS 1.2 or later for clients. Mutual TLS between cluster nodes.

Every call checked and logged

A token on every request, a policy on every path, and an audit record before any secret leaves.

Three Keys Stand Between the Disk and Your Data



A pod restart wipes the in-memory zone, and the node comes back sealed.
Three key holders unseal it again, or auto-unseal does it through your KMS or HSM.

Secrets Manager starts sealed. A sealed server can read its storage, but it cannot decrypt any of it. Each key in the chain unlocks the next one.

Key	What it protects	Where it lives
Unseal key	The root key	Nowhere. Secrets Manager splits it into 5 shares at `vault operator init` and shows each share one time only
Root key	The keyring	In storage, encrypted by the unseal key
Keyring	Every secret, token and policy	In storage, encrypted by the root key

Three key holders unseal the server by each entering one share. Secrets Manager rebuilds the unseal key in memory, then decrypts the root key and the keyring. These keys live in memory only. A pod restart wipes them, so a restarted node comes back sealed.

Lose three shares and the data is gone

Nobody can decrypt a sealed server without 3 of the 5 shares, including AccuKnox support. Give each share to a different person, and keep the holders in more than one location.

https://help.accuknox.com/secrets-manager/security/#shamirs-secret-sharing-needs-any-3-of-5-shares

Shamir's Secret Sharing Needs Any 3 of 5 Shares

Secrets Manager splits the unseal key with Shamir's Secret Sharing, an algorithm Adi Shamir published in 1979 (How to Share a Secret). Any 3 of the 5 key holders can rebuild the key. Any 2 of them learn nothing about it.

The algorithm works with a curve.

1. Secrets Manager picks a random polynomial of degree 2, one less than the threshold of 3. The point where the curve crosses the y-axis is the unseal key.
2. It computes 5 points on that curve. Each point is one key share.
3. Any 3 points fix exactly one curve of degree 2, so 3 shares give back the unseal key.
4. Two points fit an endless number of curves, each crossing the y-axis somewhere else. So 2 shares reveal nothing.

Secrets Manager runs this arithmetic one byte at a time in the finite field GF(2^8). Every byte of a share is equally likely, so a share gives no hint about the key.

Three facts follow for your operations team.

- **No single person can unseal the server.** A stolen share, or one dishonest key holder, gets nothing.
- **The order does not matter.** Any 3 of the 5 holders can unseal, each from their own machine.
- **You choose the split.** Set it at install with `vault operator init -key-shares=5 -key-threshold=3`, and change it later with `vault operator rotate-keys`.

https://help.accuknox.com/secrets-manager/security/#rotate-each-key-without-downtime

Rotate Each Key Without Downtime

What you rotate	Command	Who must take part	Effect
The keyring that encrypts data	`vault operator rotate`	One operator with the right policy	Adds a new key to the keyring. New data uses the new key, and old data still decrypts. Runs once per cluster
The unseal key shares	`vault operator rotate-keys -init -key-shares=<n> -key-threshold=<t>`	A threshold of the current share holders	Issues new shares, optionally with a new split. The old shares stop working

Both rotations run while the server is unsealed and serving requests.

Seal the Server in an Emergency

Sealing wipes the root key from memory, so every request to that node fails until 3 key holders unseal it again. Seal a node when you suspect an intrusion.

```
kubectl exec vault-accuknoxsecretmanager-0 -n accuknox -- vault operator seal
```

The command seals the node you run it on, and resets any unseal already in progress. In an HA cluster, run it on every node to seal the whole cluster.

https://help.accuknox.com/secrets-manager/security/#auto-unseal-hands-the-unseal-key-to-a-kms-or-hsm

Auto-Unseal Hands the Unseal Key to a KMS or HSM

With auto-unseal, a key management service or a hardware security module (HSM) decrypts the root key at startup. A restarted pod then unseals itself, with no key holders.

Seal type	Works offline
AWS KMS	No
Azure Key Vault	No
Google Cloud KMS	No
Oracle Cloud Infrastructure KMS	No
Alibaba Cloud KMS	No
A KMIP key server	Yes, when the key server runs inside your network
An HSM over PKCS#11	Yes. Needs the HSM build of the server image
The Transit engine of a second Secrets Manager	Yes, when the second instance runs inside your network

With auto-unseal, `vault operator init` returns recovery keys instead of unseal keys. Recovery keys authorize sensitive operations, such as generating a new root token. They cannot decrypt the root key.

The KMS key becomes the single point of recovery

If the KMS key or HSM key is unavailable, the server stays sealed. If someone deletes it, no backup can restore the data. Protect that key with the same controls as the data.

https://help.accuknox.com/secrets-manager/security/#data-on-disk-stays-encrypted

Data on Disk Stays Encrypted

The security barrier encrypts every object before it leaves the server, with AES-256 in Galois Counter Mode (GCM) and a random 96-bit nonce for each object. On every read, Secrets Manager checks the GCM authentication tag, so a changed byte on disk fails the read instead of returning altered data.

Data	Where it lives	Protection
Secrets, tokens, policies, leases	The Raft storage volume on each pod, 10Gi by default	AES-256-GCM through the security barrier
Raft snapshots	Your S3-compatible bucket	The same encrypted data. A restore needs the unseal keys
Audit log	Its own volume, 10Gi by default	Secret values are stored as salted HMAC-SHA256 hashes, never as plaintext

Data you encrypt for your own applications through the Transit engine uses `aes256-gcm96` keys by default. Transit rotates a key without re-encrypting old data, and old ciphertext still decrypts.

Disk encryption from your StorageClass adds a second layer under Secrets Manager. It protects against a stolen disk, and Secrets Manager does not depend on it.

TLS Protects Every Connection

Connection	Protection
Application or person to Secrets Manager, port 8200	TLS. The listener accepts TLS 1.2 or later by default
Node to node in an HA cluster, port 8201	Mutual TLS. Nodes exchange certificates when they join, and the cluster rotates them on a schedule
Server to its Raft storage	Local disk. All data is encrypted before write

Set `tls_cert_file`, `tls_key_file` and, if needed, `tls_min_version` in the listener block of the server configuration. The Deployment Guide uses `http://localhost:8200` through a `kubectl` port-forward, and that traffic stays inside the tunnel. Turn on TLS before an application in another pod, cluster or VM connects.

https://help.accuknox.com/secrets-manager/security/#each-caller-reads-only-its-own-paths

Each Caller Reads Only Its Own Paths

Every request carries a token, except a login request. A pod logs in with its Kubernetes service account token, a VM or job with an AppRole or a TLS certificate, and a person with OIDC, LDAP, or a username and password.

Secrets Manager then issues a short-lived token tied to one or more policies. A token reads only the paths its policies allow. A token issued before a policy change keeps its old policies until the holder logs in again.

On a managed instance, each tenant gets its own namespace, with its own permissions, resources and configuration. A shared instance includes one namespace, and a dedicated instance supports several.

Retire the root token after you create the first admin user. See Sharing Secrets in an Organisation to write least-privilege policies.

Every Request Lands in the Audit Log First

When an audit device is on, Secrets Manager writes the request and the response to the audit log before the client receives any secret. Turn on `server.auditStorage` to keep the log on its own volume.

Send the log to your SIEM through one of four audit devices.

Audit device	Sends the log to
`file`	A file on the audit volume, for a log shipper to collect
`syslog`	The local syslog agent
`socket`	A TCP, UDP or Unix socket
`http`	An HTTP endpoint

A managed dedicated instance has audit logs turned on, and it sends them straight to your AccuKnox SIEM and CDR subscription.

https://help.accuknox.com/secrets-manager/security/#what-the-security-model-covers

What the Security Model Covers

Covered	Outside the model
Eavesdropping on client, node-to-node and storage traffic	An attacker with full write control of the storage volume, who can delete or roll back data
Tampering with data at rest or in transit, which fails the integrity check	An attacker who can read the memory of a running server
Any request without a valid token and policy	Code execution on the host that runs Secrets Manager
Any request without an audit record, when an audit device is on	A compromised client that holds a valid token
Reading secrets from a stolen disk or snapshot without the unseal keys	Flaws in external systems, such as a database that a secret engine manages

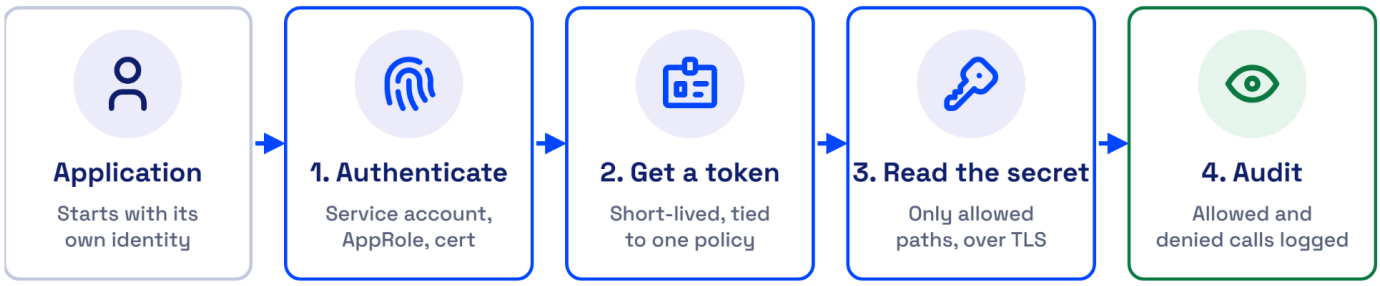
Three more limits come from where a secret goes after Secrets Manager hands it out.

- **A value the application already read.** A secret in application memory, or in a Kubernetes secret synced by the External Secrets Operator, sits outside Secrets Manager. Turn on etcd encryption in your cluster to protect a synced Kubernetes secret.
- **A snapshot plus 3 unseal shares.** Together they decrypt every secret. Give the snapshot bucket and the key shares to separate owners.
- **A lost seal.** A lost KMS key, or 3 lost shares, makes the data unrecoverable. Backups do not help.

https://help.accuknox.com/secrets-manager/rest-api/

REST API Reference

Every action in the Secrets Manager UI and CLI is an HTTPS call to the REST API. Send each call to `https://<secrets-manager-host>:8200/v1/<path>` with your token in the `X-Vault-Token` header. The API is compatible with the HashiCorp Vault HTTP API, so Vault client libraries and scripts work unchanged.



The app never holds a long-lived password. A revoked token stops access at once.

Download the Full API Document From Your Server

Your own server is the full API reference. Secrets Manager generates an OpenAPI 3 document that lists every path mounted on it, with the mount names you chose. The document covers the paths your token can reach, so a scoped token returns a scoped document.

```
curl -s --header "X-Vault-Token: <token>" \
  "$VAULT_ADDR/v1/sys/internal/specs/openapi" -o secrets-manager-openapi.json
```

Load `secrets-manager-openapi.json` into Swagger UI, Postman or any OpenAPI tool to browse and try each endpoint. Three extensions mark special paths.

Extension	Meaning
`x-vault-sudo`	The path needs `sudo` in the policy
`x-vault-unauthenticated`	The path needs no token
`x-vault-create-supported`	The path creates new items as well as updating them

https://help.accuknox.com/secrets-manager/rest-api/#authenticate-then-send-the-token

Authenticate, Then Send the Token

A call without a token fails, except for the health, seal status and login endpoints. Log in with the auth method that fits the caller, then read `auth.client_token` from the response.

```
curl -s --request POST \
  --data '{"role_id": "<role-id>", "secret_id": "<secret-id>"}' \
  "$VAULT_ADDR/v1/auth/approle/login"
```

Send the token on every later call.

```
curl -s --header "X-Vault-Token: <client-token>" \
  "$VAULT_ADDR/v1/secret/data/demo/mysql"
```

The token expires after its time to live. Log in again, or renew the token, before it expires.

The Endpoints You Will Use Most

Each path below starts with `/v1/`. A mount path such as `secret/` or `transit/` is the path you chose when you enabled the engine.

Server status | Auth | KV version 2 | Transit | TOTP | Policies

Method	Path	Does
`GET`	`sys/health`	Returns the health of the node. Load balancers use it, and it needs no token
`GET`	`sys/seal-status`	Shows whether the node is sealed and the unseal progress
`POST`	`sys/unseal`	Submits one unseal key share
`POST`	`sys/seal`	Seals the node. Needs root privileges
`GET`	`sys/storage/raft/snapshot`	Downloads a Raft snapshot

A `LIST` call also works as `GET` with `?list=true`, for clients that cannot send a custom method.

https://help.accuknox.com/secrets-manager/rest-api/#each-error-code-points-to-its-fix

Each Error Code Points to Its Fix

Status	Meaning	Fix
`400`	The request body is wrong	Check the JSON field names
`403`	The token is missing, expired, or its policy does not allow the path	Log in again, or add the path to the policy
`404`	Nothing exists at the path	Check the mount path with `GET /v1/sys/mounts`
`429`	The node hit a rate limit	Wait, then retry
`503`	The node is sealed	Unseal it with 3 of 5 key shares

A managed shared instance has API request limits. A managed dedicated instance has none. See the deployment models on accuknox.com.

Use a Client Library Instead of Raw Calls

Any HashiCorp Vault client library works against Secrets Manager. See Direct SDK and REST API for Python, Java, .NET and Node.js examples.

https://help.accuknox.com/secrets-manager/high-availability/#enable-ha-mode-with-three-replicas

Enable HA Mode With Three Replicas

Install the chart with HA and Raft storage turned on.

```
helm upgrade --install vault . -n accuknox \
  --set server.ha.enabled=true \
  --set server.ha.raft.enabled=true \
  --set server.ha.replicas=3
```

Bring the cluster up in three steps.

1. Initialize and unseal the first node, `vault-accuknoxsecretmanager-0`, as the Deployment Guide shows.
2. Join each other node to the first one. A joining node must be uninitialized, and the join uses the API address of the leader.

   ```
   kubectl exec vault-accuknoxsecretmanager-1 -n accuknox -- \
     vault operator raft join http://vault-accuknoxsecretmanager-0.vault-accuknoxsecretmanager-internal:8200
   ```

3. Unseal each joined node with 3 of the same 5 key shares. The join completes only after the unseal, and then the node receives a copy of the data.

   ```
   kubectl exec vault-accuknoxsecretmanager-1 -n accuknox -- vault operator unseal <unseal-key-1>
   ```

Repeat steps 2 and 3 for `vault-accuknoxsecretmanager-2`. With auto-unseal, a joined node unseals itself. A node joins one cluster only, and it cannot move to a different cluster later.

Run each pod on its own host. The chart leaves `server.affinity` empty, so set a pod anti-affinity rule on the hostname. The PodDisruptionBudget, on by default in HA mode, stops node maintenance from taking down a majority.

https://help.accuknox.com/secrets-manager/high-availability/#restore-from-a-snapshot

Restore From a Snapshot

Load the snapshot into a running cluster, then unseal it with the original unseal keys.

```
kubectl exec vault-accuknoxsecretmanager-0 -n accuknox -- vault operator raft snapshot restore <snapshot-file>
```

A snapshot holds encrypted data. The file alone exposes no secrets, and a restore needs the unseal keys.

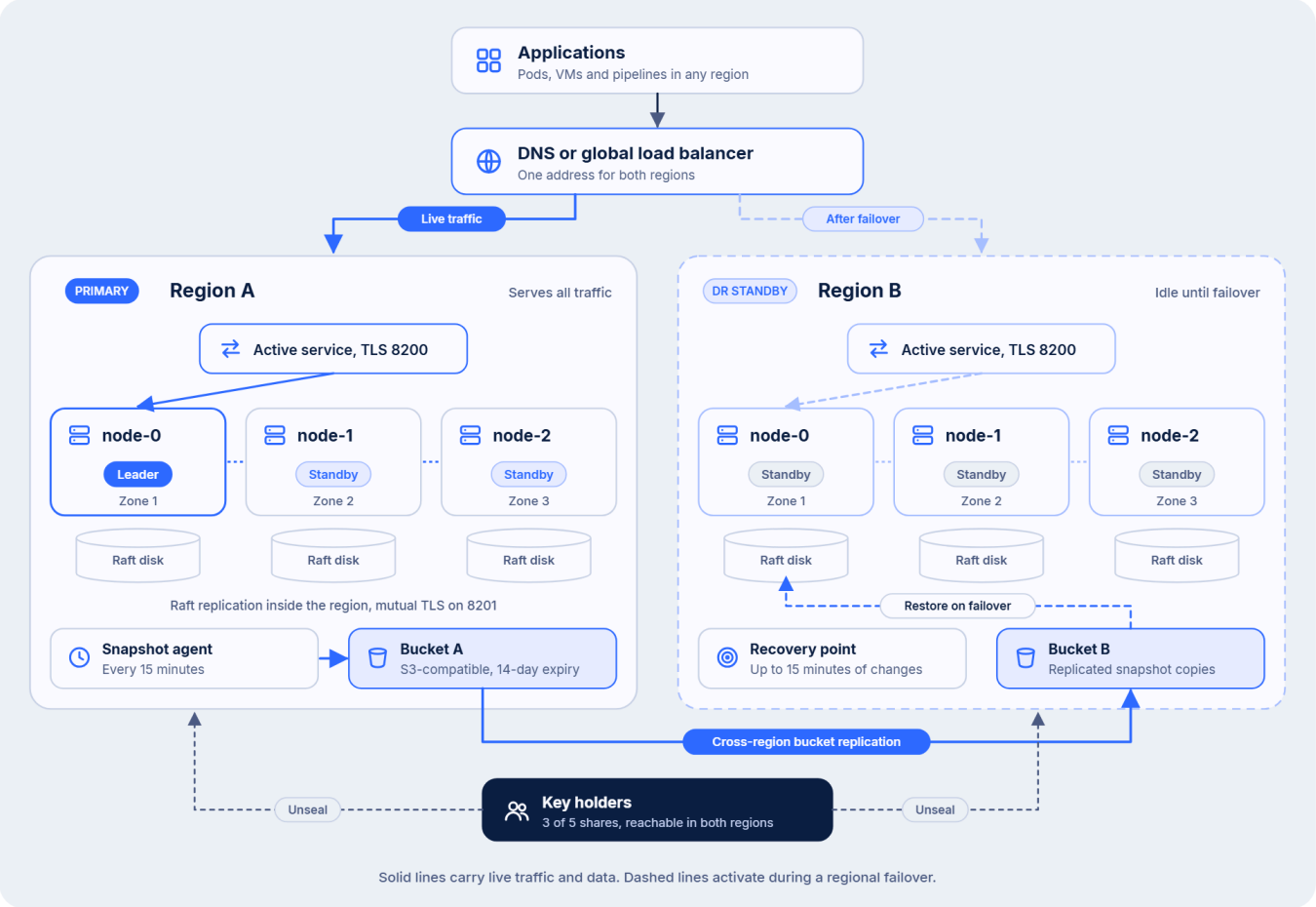
Add `-force` when the target cluster was initialized with different unseal keys from the snapshot, such as a new cluster in a DR region. The flag skips the check that the keys match the snapshot. After the restore, the cluster uses the keys from the snapshot.

```
kubectl exec vault-accuknoxsecretmanager-0 -n accuknox -- vault operator raft snapshot restore -force <snapshot-file>
```

https://help.accuknox.com/secrets-manager/high-availability/#a-second-region-recovers-from-the-loss-of-the-first

A Second Region Recovers From the Loss of the First

Run one three-node cluster in each region. The primary region serves all traffic. The disaster recovery (DR) region holds a standby cluster and a copy of every snapshot, and it takes over when the primary region fails.



Failure	What keeps Secrets Manager up	Data loss
One pod or one host	The other two nodes elect a leader	None
One availability zone	One node per zone leaves two nodes running	None
The whole primary region	The DR region restores the latest snapshot	Up to 15 minutes, with the default snapshot schedule

Your recovery time is the time your runbook takes to restore, unseal and switch DNS. Rehearse the runbook to learn that number for your environment.

Spread One Cluster Across Zones, Never Across Regions

Place the three nodes of a cluster in three availability zones of one region. Add a topology spread constraint on `topology.kubernetes.io/zone` next to the pod anti-affinity rule on the hostname.

Never stretch one Raft cluster across two regions. Every write waits for a majority of nodes, so a cross-region node adds the network round trip to each write. With two regions, one region holds two of the three nodes, and losing that region loses the majority.

Snapshots carry the data between regions. Raft replication runs inside one cluster only, so each region runs its own cluster.

https://help.accuknox.com/secrets-manager/high-availability/#prepare-the-dr-region-before-you-need-it

Prepare the DR Region Before You Need It

1. Install the chart in the DR region with HA turned on, as Enable HA Mode With Three Replicas shows.
2. Replicate the snapshot bucket to the DR region with your object storage, such as S3 cross-region replication or MinIO site replication.
3. Place at least 3 of the 5 key holders where they can reach the DR region during an outage.
4. Point applications at one DNS name or global load balancer address, never at a regional address.
5. Rehearse the failover once each quarter, and record the time it takes.

Fail Over to the DR Region

1. Confirm that the primary region cannot serve requests.
2. Initialize and unseal the DR cluster, if it is not already running.
3. Restore the latest snapshot from the replicated bucket. Use `-force`, because the DR cluster was initialized with its own keys.

```
kubectl exec vault-accuknoxsecretmanager-0 -n accuknox -- vault operator raft snapshot restore -force <snapshot-f
```

4. Unseal each DR node with 3 of the 5 original unseal key shares. The restore brings back the original keys.
5. Check the cluster with `vault status` and `vault operator raft list-peers`.
6. Switch the DNS record or the global load balancer to the DR region.

Secrets changed after the last snapshot are lost. Applications that cached a value keep working during the switch, and new logins wait until the DR region serves traffic.

Fail Back to the Primary Region

1. Rebuild the cluster in the primary region.
2. Take a fresh snapshot from the DR cluster.
3. Restore that snapshot into the primary cluster, then unseal each node with the original key shares.
4. Switch the DNS record or the global load balancer back to the primary region.
5. Return the DR cluster to standby.

> ⚠ **Do not run both regions at once**
>
> Two clusters that both accept writes drift apart, and no tool merges them. Keep the DR cluster out of the DNS record until you fail over.

https://help.accuknox.com/secrets-manager/high-availability/#six-routine-jobs-after-go-live

Six Routine Jobs After Go-Live

Job	What to do
Unseal after a restart	Any 3 of the 5 key holders unseal the node
Check backups	Confirm the snapshot CronJob succeeds and the bucket holds 14 days of files
Watch health	Turn on `serverTelemetry.serviceMonitor`, `prometheusRules` and `grafanaDashboard` for seal and HA status
Keep the audit log	Turn on `server.auditStorage`, then send the log to your SIEM with the `file`, `syslog`, `socket` or `http` audit device
Upgrade one pod at a time	The StatefulSet uses the `OnDelete` update strategy. Delete the standby pods first, then the leader
Keep access tight	Retire the root token after you create the first admin. Revoke a token under **Access → Tokens**

Two Raft commands help with node problems. `vault operator raft list-peers` shows the cluster members, and `vault operator raft remove-peer <node-id>` drops a dead node.

https://help.accuknox.com/secrets-manager/support/

Support and SLA

AccuKnox answers a P1 ticket within 1 hour on the Gold and Platinum support packages, and within 4 hours on Silver. Your support package, named in your order, sets the coverage hours and the response time for each severity. The terms come from Attachment A of the AccuKnox Subscription License Agreement.

Response Times by Severity and Package

Severity	Silver (Standard) first response	Silver updates	Gold (Premium) first response	Gold updates	Platinum (PremiumPlus) first response	Platinum updates
P1	4 hours	1 business day	1 hour	2 hours	1 hour	2 hours
P2	1 business day	2 business days	2 hours	8 hours	2 hours	8 hours
P3	2 business days	3 business days	4 hours	24 hours	4 hours	24 hours
P4	2 business days	3 business days	8 hours	48 hours	8 hours	48 hours

Feature	Silver (Standard)	Gold (Premium)	Platinum (PremiumPlus)
Coverage hours	9 hours a day, 5 days a week	12 hours a day, 6 days a week	24 hours a day, 7 days a week
Phone support	9x5	12x6	24x7
Training	Live admin webinar series and getting started guides	Standard training plus 2 seats in the premium training class	Standard training plus 5 seats in the premium training class
Adoption services	None	None	A named Customer Success Manager

Pick the Severity That Matches the Impact

Severity	Definition	Secrets Manager example
P1	A service failure or severe degradation. Users cannot reach a critical business application	The production cluster is sealed or down, and applications cannot read secrets
P2	A partial failure or mild degradation, a non-production instance down, or a bug with significant impact on an integration	One node is down, or the non-production instance is unavailable
P3	A minor impact, a non-critical bug, or trouble integrating a new application	A new application cannot log in through its auth method
P4	A how-to question or a feature request	A question about a policy, or a request for a new auth method

https://help.accuknox.com/secrets-manager/support/#open-a-ticket-by-email

Open a Ticket by Email

Send every support request to support@accuknox.com. AccuKnox issues a ticket number for each incident, and every follow-up call must quote it.

Name one support contact and one backup in your organisation. AccuKnox answers support requests from those two people only.

Put these details in the first email, so support can start at once.

1. The Secrets Manager version and the Helm chart version.
2. The output of `vault status` from each pod.
3. The output of `kubectl get pods -n accuknox`, and `kubectl describe` for any failing pod.
4. The pod logs from the time of the failure, with secret values removed.

Never send key material

Keep unseal key shares, recovery keys, the root token and secret values out of every ticket. AccuKnox support never needs them.

Response Times Exclude These Delays

AccuKnox does not count a delay that one of these causes against the response times.

- An available update that you did not install.
- A change to the operating system, network or environment made by you or a third party.
- A customization of the product.
- Use of the product in a way it was not designed for.
- Use with software or hardware AccuKnox has not approved, or on a platform outside the support matrix.

Uptime Depends on Where Secrets Manager Runs

Deployment	Who runs it	Uptime
Self-hosted with the Helm chart	Your team	You control it. A three-node cluster survives the loss of one node, and a second region survives the loss of the first. See High Availability and Backup
Managed shared or dedicated instance	AccuKnox	Every managed instance runs in high availability mode. Your order names the agreement that sets the uptime commitment

For the AccuKnox Cloud Service, the Subscription License Agreement commits to 99.9% availability over each year. The SaaS SLA adds service credits when monthly uptime falls below 99.9%. Check live status at status.accuknox.com.

AccuKnox professional services can also review your application and make the code change to read secrets through the SDK.

https://help.accuknox.com/secrets-manager/air-gapped/#5-unseal-with-your-own-key-holders

5. Unseal With Your Own Key Holders

The default seal splits the root key into 5 Shamir key shares, and any 3 unseal the server. No cloud key service is involved, so the unseal works fully offline. Give the shares to different people inside your organisation.

Auto-unseal also works offline, through a key service inside your network.

Offline auto-unseal	What it needs
An HSM over PKCS#11	The HSM build of the server image, and the HSM vendor's PKCS#11 library
A KMIP key server	A KMIP server reachable from the cluster
The Transit engine of a second Secrets Manager	A second Secrets Manager instance inside the network

The auto-unseal example in `values.yaml` uses Google Cloud KMS, which needs internet access. See Security and Encryption for every seal type.

https://help.accuknox.com/secrets-manager/external-secrets/#trust-an-application-cluster-that-runs-elsewhere

Trust an Application Cluster That Runs Elsewhere

Secrets Manager checks each login by asking the application cluster to review the service account token. It needs a reviewer account in that cluster, plus the cluster address and CA certificate.

1. In the application cluster, create a reviewer service account and let it review tokens.

```
kubectl create serviceaccount secrets-manager-reviewer -n external-secrets
kubectl create clusterrolebinding secrets-manager-reviewer \
  --clusterrole=system:auth-delegator \
  --serviceaccount=external-secrets:secrets-manager-reviewer
```

2. Create a long-lived token for the reviewer account.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: secrets-manager-reviewer-token
  namespace: external-secrets
  annotations:
    kubernetes.io/service-account.name: secrets-manager-reviewer
type: kubernetes.io/service-account-token
EOF
```

3. Read the token, the CA certificate and the API server address.

```
kubectl get secret secrets-manager-reviewer-token -n external-secrets -o jsonpath='{.data.token}' | base64 -d > r
kubectl get secret secrets-manager-reviewer-token -n external-secrets -o jsonpath='{.data.ca\.crt}' | base64 -d >
kubectl cluster-info
```

4. In Secrets Manager, enable the Kubernetes auth method at its own path and point it at the application cluster.

```
vault auth enable -path=k8s-demo-cluster kubernetes
vault write auth/k8s-demo-cluster/config \
  token_reviewer_jwt=@reviewer.jwt \
  kubernetes_host=https://<application-cluster-api-server>:443 \
  kubernetes_ca_cert=@ca.crt
```

5. Create the role that binds the operator's service account to the policy.

```
vault write auth/k8s-demo-cluster/role/demo-access \
  bound_service_account_names=external-secrets \
  bound_service_account_namespaces=external-secrets \
  policies=demo-access \
  ttl=1h
```

Use one auth path per application cluster, such as `k8s-prod-eu` and `k8s-prod-us`. Each path then trusts one cluster only.

https://help.accuknox.com/secrets-manager/sdk-integration/

Third-party clients

These libraries are third-party Vault clients, not AccuKnox software. Their maintainers publish the releases and the documentation.

https://help.accuknox.com/faqs/secrets-manager/

Secrets Management

1. What is AccuKnox Secrets Manager and how is it different from Vault?

2. What secret engines and authentication methods are supported?

3. Can existing Vault applications work with AccuKnox Secrets Manager without code changes?

4. How are dynamic secrets and secret rotation handled?

5. What deployment models and access controls are available?

6. Which Kubernetes clusters does Secrets Manager run on?

7. Should I use the SDK or the External Secrets Operator?

8. Does an application pick up a changed secret on its own?

9. Can Secrets Manager run with no internet access?

10. How does Secrets Manager protect secrets at rest?

Secrets Manager **encrypts every secret before it writes it to disk**. A chain of three keys protects the data.

- **Encryption key**: Encrypts the secrets.
- **Root key**: Encrypts the encryption key.
- **Unseal key**: Decrypts the root key. Shamir's Secret Sharing splits it into 5 shares, and any 3 unseal the server.

Secrets Manager never stores the unseal key. See Security and Encryption.

11. Is there a REST API?

Yes. Every UI and CLI action is a call to `https://<host>:8200/v1/<path>`, compatible with the HashiCorp Vault HTTP API. Your server also generates an **OpenAPI 3 document** of every path your token can reach. See the REST API Reference.

12. Can Secrets Manager run across regions with disaster recovery?

Yes. Run a **three-node cluster in each region**, and replicate the snapshot bucket to the second region. With the default 15-minute snapshot schedule, a regional failover loses at most 15 minutes of secret changes. See High Availability and Backup.

13. What support SLA comes with Secrets Manager?

Your support package sets the response time. A **P1 ticket gets a first response within 1 hour** on Gold and Platinum, and within 4 hours on Silver. Platinum covers 24 hours a day, 7 days a week. Send requests to support@accuknox.com. See Support and SLA.